

Engineering Reboot 2026

Day 20 Field Manual

Version 1.0 - Week 4 Baseline

Topic	Modules & Imports
Objective	Learn to organize Python applications into reusable modules and import functionality across multiple source files.
Expected Outcome	Build a modular calculator application that separates business logic from the user interface.
Publication Source	Formatted from the approved Day 20 manuscript source.

MISSION BRIEF

Today introduces modular Python programming. The objective is to move from single-file scripts into small multi-file applications where responsibilities are separated clearly.

CONCEPTS

- Creating Python modules
- import statement
- from ... import ...
- Reusable code organization
- Separation of concerns
- Introduction to `__name__` and `__main__`

A Python module is simply a .py file that can be imported by another Python program. As programs grow, modules help keep related functions together and prevent one source file from becoming difficult to maintain.

EXERCISE 01

Filename: `math_utils.py`

Objective: Create a reusable mathematics module.

Requirements

- Implement `add()`, `subtract()`, `multiply()`, and `divide()`.
- Each function accepts two numeric parameters.
- Each function returns a value.
- Do not use `input()` or `print()` inside the module.

Expected Behavior

The functions can be imported and reused by other programs.

Suggested Function Interface

```
def add(a, b):  
    return a + b  
  
def subtract(a, b):  
    return a - b  
  
def multiply(a, b):  
    return a * b  
  
def divide(a, b):  
    return a / b
```

EXERCISE 02

Filename: `import_demo.py`

Objective: Import and use the reusable functions created in `math_utils.py`.

Requirements

- Import all mathematical functions.
- Execute each function using sample values.
- Display the returned results.

Expected Behavior

The program demonstrates successful module imports and correct function execution.

Expected Output Example

Addition: 15
Subtraction: 5
Multiplication: 50
Division: 2

PROJECT / IMPLEMENTATION ASSIGNMENT

Filenames: calculator_module.py, main.py

Objective: Separate calculator business logic from the user interface.

calculator_module.py	main.py
Contains calculation functions only. No input(), print(), menu logic, or user interaction.	Displays the menu, reads input, calls imported functions, displays output, and controls application flow.

Functional Requirements

- Display a calculator menu.
- Accept two numeric inputs.
- Perform addition, subtraction, multiplication, and division.
- Import calculation functions from calculator_module.py.
- Continue processing until the user selects Exit.

Suggested Menu

```
=====
ENGINEERING CALCULATOR
=====
1. Add
2. Subtract
3. Multiply
4. Divide
5. Exit
```

Acceptance Criteria

- User interface resides in main.py.
- Mathematical logic resides in calculator_module.py.
- The application produces correct results and exits cleanly.

VALIDATION MATRIX

Validation Item	Pass / Fail
Application launches successfully	☐
Module imports correctly	☐
Functions execute correctly	☐
Addition, subtraction, multiplication and division return correct results	☐
Menu displays correctly	☐
User can perform multiple calculations	☐
Application exits cleanly	☐
Business logic remains inside calculator_module.py	☐
User interface remains inside main.py	☐

ENGINEERING DISCUSSION

Today's lesson introduces modular software design. Rather than placing every function inside a single source file, responsibilities are separated into reusable modules. This improves readability, maintainability, testing, and future extensibility. The user interface becomes independent of the calculation engine, allowing other applications to reuse the same module.

STRETCH GOAL

Extend `calculator_module.py` by implementing `power()`, `modulus()`, and `floor_division()`. Update `main.py` to expose these operations through new menu options while preserving the modular architecture.

CHALLENGE ENHANCEMENT

Create `scientific_calculator.py` that imports the existing `calculator_module.py` without modifying it. Demonstrate that one reusable module can support multiple applications.

DELIVERABLES

Exercises	Project	Documentation
<code>math_utils.py</code> <code>import_demo.py</code>	<code>calculator_module.py</code> <code>main.py</code>	<code>Notes.md</code> <code>Journal.md</code>

EVIDENCE CHECKLIST

- Code Evidence for every Python source file
- Program Output Evidence
- `Notes.md` updated
- `Journal.md` updated

SOURCE CONTROL

- Commit completed
- Push completed
- Remote synchronization verified

PREVIEW OF NEXT TOPIC

Week 5 begins Professional Python Foundations. Upcoming topics include Exception Handling, Input Validation, Defensive Programming, Type Hints, and Program Organization. These concepts will build on the modular design introduced today and prepare larger multi-file applications.

EDITORIAL EXPANSION - DAY 20

Implementation Notes

A module should contain reusable functionality only. Avoid placing `input()`, `print()`, or menu logic inside reusable modules. Returning values instead of printing them allows the same functions to be reused by command-line applications, graphical interfaces, automated tests, and future web services.

Example Module Design

`math_utils.py` should expose a clean public interface consisting of `add()`, `subtract()`, `multiply()`, and `divide()`. Each function should perform one operation and return the result. Keeping functions small makes them easier to test and debug.

Example Program Flow

1. `main.py` displays the calculator menu.
2. User selects an operation.
3. User enters two numbers.
4. `main.py` calls the corresponding function in `calculator_module.py`.
5. Returned value is displayed.
6. Control returns to the menu until Exit is selected.

Expected Output Example

```
=====
ENGINEERING CALCULATOR
=====
1. Add
2. Subtract
3. Multiply
4. Divide
5. Exit

Select Option: 1
Enter first number: 12
Enter second number: 8

Result: 20
```

Engineering Commentary

Today's exercise introduces the architectural principle of separation of concerns. The user interface is responsible for interacting with the user, while the calculation module is responsible only for computation. This separation allows either component to evolve independently and is representative of professional software engineering practices.

Professional Practice

As projects grow, modules are commonly grouped into packages. Although today's lesson uses only two or three source files, the same principles scale to large applications consisting of dozens of modules. Consistent naming, single-purpose modules, and clear interfaces reduce maintenance costs and improve readability.

Self-Assessment

- Can I explain why modules are preferable to one large source file?
- Can I create a reusable function without using `input()` or `print()`?
- Can I import an entire module and individual functions?
- Can I identify which code belongs in `main.py` versus `calculator_module.py`?